

So where does the money actually go?

- Continuous batching is the highest-return change on a self-hosted stack, commonly delivering two to three times the throughput of static batching.
- Quantization increases the batch size that fits in a given GPU, and the only trustworthy way to judge the quality cost is your own evaluation set.
- Spot capacity suits evaluation runs and batch scoring, and suits interactive serving poorly, because reloading tens of gigabytes of weights after a preemption is not recoverable within a user's patience.
- Scale to zero fails for the same reason, so keeping one replica warm per model with daytime traffic usually beats saving a few dollars an hour.

Two newer patterns are worth tracking. Disaggregated serving splits prefill and decode into independently scaled pools, and prefix-aware routing sends each request to the replica already holding a matching cached prefix. The Kubernetes Gateway API Inference Extension does exactly that, scoring endpoints on queue depth and cache locality instead of round robin.

Why should you route before you generate?

The cheapest inference is the one you do not run.

Routing needs three things: a classifier or rule set, an evaluation set proving cheaper paths hold quality, and observability to catch drift. Without the evaluation set, routing is a quality regression you have not measured yet.

Cache at several levels

Caching is not one technique. It is four, with different hit profiles and different failure modes.

Cache layer	What is stored	Where it helps	Main risk
Exact response	Output keyed on prompt hash	High-repetition workloads	Stale answers when data changes
Semantic	Output keyed on embedding similarity	Paraphrased questions	False hits when threshold is loose
Provider prefix cache	Repeated static prompt prefix	Long system prompts, few-shot examples	Needs stable prompt ordering
KV prefix reuse	Attention state on the replica	Multi-turn conversations	Needs cache-aware routing

Semantic caching fails in the most interesting ways, because

similarity is not equivalence and small wording changes carry large intent changes. Set the threshold conservatively, scope keys by tenant, and log near misses so you can tune with evidence.

What should FinOps actually measure?

Cost per token is an accounting metric. It falls when you switch to a cheaper model even if that model needs three attempts to get the answer right. Cost per successful task captures retries, escalations, and quality failures in one number.

- Tag every request at the gateway with team, use case, model, and environment.
- Design those tags before the first production call, since they cannot be added retroactively without breaking continuity.
- Alert on rate of spend rather than monthly totals, because a runaway agent loop can burn a month's budget long before a cumulative threshold fires.

How do you ship model and prompt changes safely?

LLM platforms need three delivery tracks running at different cadences against one declared state.

What changes is the promotion gate:

- Alongside tests and security scanning, the pipeline blocks on evaluation thresholds, projected cost per task, and canary quality signals.
- Shadow traffic compares two model versions on identical production inputs without exposing users to either result.
- Canary by request class beats canary by percentage, because routing all queries of one low-risk type reveals more than five percent of everything.
- Rollback needs the previous model pinned and warm, with the prompt versioned alongside it, since the pair was evaluated together.

What changes when you run it in production?

Conventional high availability assumes you can add a replica. With accelerators, redundancy means paying for an idle GPU, so multi-provider fallback at the gateway is usually cheaper insurance than duplicate capacity, provided the path is exercised. Untested failover is a hypothesis, not a control.

A degradation ladder, agreed in advance and implemented at the gateway, converts a hard outage into a slower service.

Level	Trigger	Behaviour
1	Queue wait above target	Reduce max output tokens, trim retrieval k
2	Primary model degraded	Route to secondary provider or smaller model
3	Sustained capacity shortfall	Serve cached answers for common intents
4	Inference unavailable	Disable AI paths, expose underlying tools directly